OPERATING SYSTEMS ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named CSA04-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHubRepositoryLink:https://github.com/192511225simatscode/operatingsystem.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>


#define N 4


int main(void)
{
   const char *name[N] = {"P1", "P2", "P3", "P4"};     int arrival[N] = {0, 1, 2, 3};
int burst[N] =
{8, 4, 9, 5};     int quantum = 3;


   int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};     for (int i = 0; i < N; i++) left[i] =
burst[i];     int time = 0, done = 0;     queue[tail++] = 0;     added[0] = 1;     printf("Gantt:");     while (done < N)
{          int p = queue[head++];
       // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
       //   advance time, reduce left[p]; add every process that has now arrived and is          //   not yet added to the queue; then put p back
at the end of the queue if it       //   still has time left, otherwise record finish[p] and count it as done


   }
   double tw = 0, tt = 0;     printf("\nProcess  TAT  WT\n");     for (int i = 0; i < N; i++) {         int
tat = finish[i] - arrival[i], wt = tat - burst[i];         tt += tat;         tw += wt;         printf("%-7s %4d
%3d\n", name[i], tat, wt);
   }
   printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);     return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)
Process  TAT  WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-toone and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>


/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */ int main(void)
{
   int fd[2];     if (pipe(fd) == -1) { perror("pipe"); return 1; }
pid_t pid = fork();     if (pid == 0) {         close(fd[1]);
/* child only reads */         int r, count = 0, sum = 0, max = 0;
       // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent        //   closed the pipe); keep the count, the sum and the
maximum


       printf("analyser: %d readings, average %.1f, max %d\n", count,                count ? (double)sum /
count : 0.0, max);         close(fd[0]);         fflush(stdout);          _exit(0);
   }
   close(fd[0]);                        /* parent only writes */     int readings[] = {31, 33, 36, 34, 41, 38};     for (int i = 0; i < 6; i++)         if
(write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");     close(fd[1]);                          /* signals end of data to the
child */     waitpid(pid, NULL, 0);     printf("reader: done\n");     return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41 reader:
done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

> A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>


#define SIZE 5
#define ITEMS 20


int buffer[SIZE], in = 0, out = 0; sem_t empty, full; pthread_mutex_t lock =
PTHREAD_MUTEX_INITIALIZER; long consumed_sum = 0;


void *producer(void *arg) {
   (void)arg;      for (int item = 1; item <= ITEMS; item++) {
      // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in         //   circularly, unlock, then post full


   }      return NULL; }


void *consumer(void *arg) {
   (void)arg;      for (int k = 0; k < ITEMS; k++) {
      // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance       //   out circularly, unlock, post empty and add the
item to consumed_sum


   }      return NULL; }


int main(void)
{
   sem_init(&empty, 0, SIZE);     sem_init(&full, 0, 0);      pthread_t p, c;     pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);      pthread_join(p, NULL);      pthread_join(c, NULL);      printf("items consumed:
%d, sum = %ld (expected %d)\n", ITEMS, consumed  sum,
         ITEMS * (ITEMS + 1) / 2);      sem_destroy(&empty);
sem_destroy(&full);      return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm 13 Marks
Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>


#define P 5
#define R 3


int avail[R] = {3, 3, 2}; int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}}; int alloc[P][R] = {{0,
1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};


/* Returns 1 if the state is safe and stores a safe sequence in seq. */ int is_safe(int seq[P])
{
   int work[R], finish[P] = {0}, count = 0;     memcpy(work, avail,
sizeof work);     while (count < P) {         int found = 0;         for (int p
= 0; p < P; p++) {             if
(finish[p]) continue;
           // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p     //   finish: add alloc[p] to work, mark finish[p],
append p to seq and set             //   found = 1


       }         if (!found)
return 0;      }
return 1; }


void request(int p, const int req[R])
{
   for (int r = 0; r < R; r++)         if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {             printf("P%d
request:
must wait (not available)\n", p);             return;
       }
   for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }      int seq[P];     if (is_safe(seq)) {
       printf("P%d request: granted, safe sequence", p);         for (int i = 0; i < P; i++)
printf(" P%d", seq[i]);         printf("\n");      } else {
       for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }         printf("P%d request: denied (unsafe)\n", p);     }
}


int main(void)
{
   int seq[P];     if (is_safe(seq)) {
       printf("initial state safe:");         for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
printf("\n");
   }
   request(1, (int[R]){1, 0, 2});     request(4, (int[R]){3, 3,
0});     request(0, (int[R]){0, 2, 0});     return 0;   }
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available) P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks

Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6
Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		50

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend

TABLE OF CONTENTS

1. Q1 – Processes and Process States
2. Q2 – CPU Scheduling
3. Q3 – Threads
4. Q4 – Inter-Process Communication
5. Q5 – Synchronisation: Critical Sections, Semaphores and Monitors
6. Q6 – Deadlock Handling and Banker's Algorithm
7. Overall Conclusion
8. References

Q1 – PROCESSES AND PROCESS STATES (6 MARKS)

1.1 Process concept and process-state model

A process is a program in execution. In an online examination server, separate activities may include student sessions, automatic answer saving and proctoring-video processing. A process moves through five standard states:

State	Meaning
New	The process is being created and its processcontrol information is prepared.
Ready	The process is in main memory and is waiting for CPU time.
Running	The CPU is currently executing the process.
Waiting / Blocked	The process is waiting for an event such as disk, network or camera I/O.
Terminated	The process has completed or has been stopped.

Process-state transition diagram:

Transition	Reason
New → Ready	The OS admits the process to the ready queue.
Ready → Running	The CPU scheduler dispatches the process.
Running → Ready	A time slice expires or a higher-priority process pre-empts it.
Running → Waiting	The process requests I/O or waits for an event.
Waiting → Ready	The required I/O or event completes.
Running → Terminated	The process exits or is terminated.

[Diagram representation] NEW → READY ⇄ RUNNING → TERMINATED

↑ ↓

└── WAITING ←┘

The arrow from RUNNING to READY represents pre-emption; RUNNING to WAITING represents an I/O wait.

1.2 Process Control Block (PCB)

The operating system stores process information in a Process Control Block. The PCB allows the OS to stop one process and later resume it correctly.

PCB field	Purpose
Process ID (PID)	Unique number identifying the process.
Process state	New, ready, running, waiting or terminated.
Program counter	Address of the next instruction to execute.
CPU registers	Saved register values during a context switch.
CPU scheduling information	Priority, queue links and scheduling parameters.
Memory-management information	Page tables, address-space and memory limits.
Accounting information	CPU time, user ID, process limits and usage.
I/O status information	Open files, allocated devices and pending I/O.

1.3 Linux process observation using fork(), ps and /proc

The fork() system call creates a child process. It returns 0 in the child, the child's PID in the parent, and −1 if creation fails. The following program creates a child, displays both process IDs, and keeps the child alive briefly so it can be inspected.

```
/* q1_fork.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
pid_t pid = fork();
   if (pid < 0) { perror("fork"); return 1; }
if (pid == 0) {
      printf("Child: PID=%ld PPID=%ld\n",
(long)getpid(),                  (long)getppid());
sleep(10);
      _exit(0);
} else {
      printf("Parent: PID=%ld Child PID=%ld\n",
           (long)getpid(), (long)pid);
char cmd[100];
      snprintf(cmd, sizeof cmd, "ps -o pid,ppid,state,comm -p %ld,%ld",
            (long)getpid(), (long)pid);
system(cmd);         char path[64];
      snprintf(path, sizeof path, "/proc/%ld/status", (long)pid);
printf("Inspect child with: cat %s\n", path);         waitpid(pid,
NULL, 0);         puts("Parent: child completed");
   }
return 0; }
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra q1_fork.c -o q1_fork
./q1_fork
```

```
# In another terminal, while the child sleeps:
ps -ef | grep q1_fork cat
/proc/<child_pid>/status
```

1.4 Evaluation for 2,000 active student sessions

Factor	Effect at high process count	Evaluation
Context switching	Frequent switching saves/restores registers and changes address-space context; cache and TLB effects may add overhead.	A very large runnable process count can reduce useful CPU time.
Memory per process	Each process has a separate virtual address space and kernel bookkeeping.	2,000 full processes consume more memory than lightweight sessions in threads or a bounded worker pool.
I/O waiting	Network, database and video I/O can leave processes blocked.	Blocked processes do not use CPU, but they still consume memory and descriptors.
Isolation	A process has its own address space.	A crash is better isolated from other sessions, improving fault containment.

A separate process per student gives strong isolation, but it is not automatically the most efficient design for 2,000 simultaneous sessions. A fixed pool of worker processes is recommended for request execution, while asynchronous I/O or a bounded number of threads can manage many mostly waiting network sessions. Critical proctoring or security components can remain isolated where necessary.

1.5 Design alternatives and complexity

Design	Advantages	Limitations	Decision
One process per student	Strong memory and fault isolation.	Higher memory use and processmanagement overhead.	Use only where isolation is essential.
One thread per student	Lower creation and switching overhead; shared server data is easy to access.	Shared address space means a serious memory error can affect the server; synchronisation is required.	Suitable for moderate workloads with careful limits.

Fixed worker-process pool	Bounded memory and predictable worker count; process isolation between workers.	Requires a queue and load balancing; a saturated pool increases queue delay.	Recommended for the examination server.

For N sessions, maintaining a pool of W workers keeps the number of execution processes approximately O(W), rather than O(N). A per-session thread design uses O(N) thread-control structures and stacks. The total work still depends on the requests; the pool limits concurrency and controls resource use.

1.6 Alternative concept: sessions as threads

If each student session were a thread in one server process, each thread would have its own stack, registers and scheduling state, while sharing the process code, data, heap and open file descriptors. Thread context switches are generally cheaper than process switches because the address space is shared. However, a segmentation fault or memory corruption in a thread can terminate or corrupt the entire server. Separate processes provide stronger isolation but require more memory. A practical compromise is a bounded thread pool inside multiple worker processes.

conclusion

The one-process-per-student model is simple and isolated but scales poorly in memory and management overhead. A bounded worker pool, optionally combined with asynchronous I/O, provides better scalability while retaining process-level fault containment.

Q2 – CPU SCHEDULING (8 MARKS)

2.1 Given workload and assumptions

The question provides four processes with the following arrival and CPU-burst times. For priority scheduling, the source PDF does not specify numeric priorities. To make the comparison reproducible, this report uses the stated illustrative assumption that a smaller priority number means higher priority: P2=1, P4=2, P1=3 and P3=4. The priority result must be recalculated if the instructor supplies different priorities.

Process	Arrival time	Burst time	Priority (assumed)
P1	0	8	3
P2	1	4	1

P3	2	9	4
P4	3	5	2

Definitions: Turnaround time (TAT) = Completion time − Arrival time. Waiting time (WT) = TAT − Burst time. Response time (RT) = First CPU start time − Arrival time.

2.2 FCFS scheduling

First Come First Served executes processes in arrival order without pre-emption. Gantt chart: 0 | P1 | 8 | P2 | 12 | P3 | 21 | P4 | 26

Process	Completion	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18
Average		15.25	8.75	8.75

2.3 SJF non-preemptive scheduling

Shortest Job First selects the shortest burst among the processes that have arrived whenever the CPU becomes free. P1 starts at time 0 and completes at 8; then P2, P4 and P3 are selected.

Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	Completion	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

2.4 SRTF pre-emptive scheduling

Shortest Remaining Time First always runs the arrived process with the least remaining CPU burst. At time 1, P2 arrives with a shorter burst than P1 and pre-empts it. P4 later becomes the shortest remaining job.

Gantt chart: 0 | P1 | 1 | P2 | 5 | P4 | 10 | P1 | 17 | P3 | 26

Process	Completion	TAT	WT	RT
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2
Average		13.00	6.50	4.25

2.5 Priority scheduling (non-preemptive; stated assumption)

Using the assumed priorities P2=1, P4=2, P1=3 and P3=4, P1 runs first because it is the only process available at time 0. At time 8, the waiting processes are selected in priority order. Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	Completion	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

2.6 Round Robin scheduling

Round Robin gives each ready process a time quantum of 3 units. The queue is rotated after each time slice. The completed program in Section 2.7 follows the arrival order and re-queues unfinished processes.

Gantt chart (quantum = 3): P1(0–3) → P2(3–6) → P3(6–9) → P4(9–12) → P1(12–15) → P2(15–16) → P3(16–19) → P4(19–21) → P1(21–23) → P3(23–26).

Process	Completion	TAT	WT	RT

P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6
Average		20.00	13.50	3.00

2.7 Completed Round Robin C program

```
/* q2_round_robin.c */
#include <stdio.h>
#define N 4 int
main(void) {
   const char *name[N] = {"P1","P2","P3","P4"};
int arrival[N] = {0,1,2,3}, burst[N] = {8,4,9,5};
int quantum = 3;
   int left[N], finish[N], first[N], queue[128], head=0, tail=0;
int added[N]={0}, time=0, done=0;
   for (int i=0;i<N;i++) { left[i]=burst[i]; first[i]=-1; }
queue[tail++]=0; added[0]=1;     printf("Gantt:");
while (done<N) {         if (head==tail) {             for
(int i=0;i<N;i++)
            if (!added[i] && arrival[i]<=time) {
               queue[tail++]=i; added[i]=1;
            }
         if (head==tail) { time++; continue; }
      }
      int p=queue[head++];
if (first[p]<0) first[p]=time;
      int start=time, run=(left[p]<quantum)?left[p]:quantum;
time+=run; left[p]-=run;
      printf(" %s(%d-%d)",name[p],start,time);
for (int i=0;i<N;i++)
         if (!added[i] && arrival[i]<=time) {
            queue[tail++]=i; added[i]=1;
         }
      if (left[p]>0) queue[tail++]=p;
      else { finish[p]=time; done++; }
   }
   double sw=0, st=0, sr=0;
printf("\nProcess TAT WT RT\n");     for
(int i=0;i<N;i++) {
      int tat=finish[i]-arrival[i], wt=tat-burst[i];
int rt=first[i]-arrival[i];
      sw+=wt; st+=tat; sr+=rt;
      printf("%-7s %3d %3d %3d\n",name[i],tat,wt,rt);
   }
   printf("Average TAT=%.2f WT=%.2f RT=%.2f\n",
st/N,sw/N,sr/N);     return 0; }
```

Compile and execute: gcc -std=c11 -Wall -Wextra q2_round_robin.c -o q2_rr && ./q2_rr

The source PDF's expected output for quantum 3 gives average TAT = 20.00 and average WT = 13.50. The response-time column above is additionally calculated from the first start times. 2.8 Effect of time quantum

Quantum	Expected scheduling behaviour	Evaluation
1	Very frequent rotation; short jobs receive quick initial service, but many context switches occur.	High scheduling overhead; response can improve, but total useful CPU time may fall.
3	Moderate rotation; source PDF supplies the expected Gantt chart and averages.	Reasonable balance for the given workload.
5	Fewer switches than quantum 3; longer uninterrupted execution.	May reduce overhead but short jobs can wait longer.
20	Approaches FCFS for this workload because each job can finish in one turn once selected.	Few switches, but interactive response and fairness are weaker.

Actual context-switch counts and timings depend on the implementation and operating system. Record the measured counts and elapsed times after running the program with each quantum; do not treat the number of printed time slices as an exact hardware context-switch count.

2.9 Starvation, ageing and multilevel feedback queue

- SJF/SRTF starvation: a long job may repeatedly wait when short jobs continue to arrive.
- Priority starvation: low-priority processes may wait indefinitely while higher-priority work remains ready.
- Ageing: gradually increase the priority of a process the longer it waits, ensuring that it eventually runs.
- Multilevel Feedback Queue (MLFQ): new or interactive jobs begin in a high-priority queue with a short quantum. Jobs that repeatedly use their full quantum move to lower-priority queues with longer quanta. Periodic priority boosts prevent starvation.

2.10 Recommendation and complexity

For a mixed video-encoding service, an MLFQ or a carefully tuned pre-emptive policy is preferable to pure FCFS. Short thumbnail jobs should receive fast initial service, while long encoding jobs

should continue to make progress in lower queues. For the small fixed set here, SRTF has the lowest average waiting time (6.50) and response time (4.25) among the calculated policies, but it requires burst-time estimates and can starve long jobs. MLFQ is a more practical adaptive recommendation when job length is not known in advance.

With a simple ready-queue implementation, selecting the shortest job by scanning n jobs costs O(n) per decision; a heap can reduce selection to O(log n). A queue-based Round Robin dispatch can be O(1) per dequeue/enqueue operation. MLFQ selection can also be O(1) when each priority queue is maintained separately, excluding queue migration and bookkeeping.

Q3 – THREADS (6 MARKS)

3.1 Thread concept and shared resources

A thread is the smallest unit of CPU execution within a process. The traffic-camera application can process frames from 64 cameras concurrently. Threads in the same process share the program code, global data, heap, open files and process resources. Each thread has its own thread ID, program counter, CPU registers and stack.

Shared by threads	Private to each thread
Code and executable instructions	Thread ID and scheduling state
Global and static variables	Program counter and register set
Heap and dynamically allocated objects	Stack and local automatic variables
Open file descriptors and process resources	Thread-specific data

3.2 Thread implementation models

Model	Description	Evaluation
Many-to-one	Many user threads map to one kernel thread.	Low management overhead, but one blocking system call can block all threads and multiple CPUs cannot execute those threads in parallel.
One-to-one	Each user thread maps to a kernel thread.	Supports parallelism and independent blocking, but many kernel threads increase management overhead.
Many-to-many	Many user threads are multiplexed over a set of kernel threads.	Balances flexibility and parallelism; implementation is more complex.

User-level threads are managed by a runtime library without requiring the kernel to schedule each user thread individually. Kernel-level threads are known to and scheduled by the operating system. Kernel-level scheduling supports multicore parallelism and independent blocking.

3.3 Parallel camera-processing design

For 64 cameras, creating 64 threads is a useful baseline, but a fixed pool of 4, 8 or 16 workers can reduce creation overhead and bound memory consumption. Each worker repeatedly obtains the next

camera/frame task from a shared work queue. The queue index and shared counters must be protected by a mutex or atomic operation.

3.4 POSIX thread program: worker pool

```
/* q3_threads.c - compare 1, 4, 8 and 16 worker threads */
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define CAMERAS 64
#define WORK 300000

static int next_camera = 0; static
long results[CAMERAS];
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg) {
   (void)arg;
for (;;) {
      pthread_mutex_lock(&lock);
int id = next_camera++;
pthread_mutex_unlock(&lock);
if (id >= CAMERAS) break;

      unsigned long x = (unsigned long)(id + 1);
for (int k=0; k<WORK; k++)
         x = (x * 1664525UL + 1013904223UL) % 4294967291UL;
results[id] = (long)(x % 1000000UL);
   }
   return NULL;
}
int main(void) {     int
pools[] = {1,4,8,16};
for (int z=0; z<4; z++) {
next_camera=0;
      for (int i=0;i<CAMERAS;i++) results[i]=0;
int n=pools[z];
      pthread_t *th=calloc((size_t)n,sizeof *th);
struct timespec a,b;
      clock_gettime(CLOCK_MONOTONIC,&a);
      for (int i=0;i<n;i++)
         if (pthread_create(&th[i],NULL,worker,NULL)!=0) {
perror("pthread_create"); return 1;
         }
      for (int i=0;i<n;i++) pthread_join(th[i],NULL);
clock_gettime(CLOCK_MONOTONIC,&b);
      double sec=(b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
long checksum=0;
      for (int i=0;i<CAMERAS;i++) checksum+=results[i];
printf("%2d threads: %.6f s, checksum=%ld\n",n,sec,checksum);
free(th);
   }
   pthread_mutex_destroy(&lock);
return 0;
}
```

Compile and run: gcc -std=c11 -O2 -Wall -Wextra -pthread q3_threads.c -o q3_threads && ./q3_threads

3.5 Speed-up and performance evaluation

Speed-up is calculated as S(p) = T(1) / T(p), where T(1) is the measured one-thread time and T(p) is the measured time with p workers.

Workers	Measured time (seconds)	Speed-up
1	Record after execution	1.00
4	Record after execution	T(1)/T(4)
8	Record after execution	T(1)/T(8)
16	Record after execution	T(1)/T(16)

Speed-up eventually stops increasing because of serial work, mutex contention, memory-bandwidth limits, cache misses, operating-system scheduling overhead and the number of physical CPU cores. If the workload is I/O-bound rather than CPU-bound, adding threads may improve overlap but will not necessarily increase CPU throughput.

3.6 Race condition in shared counters

If two threads execute counter++ simultaneously, both may read the same old value, increment it and write the same new value. One update is lost. This is a race condition. Use a mutex around the update or use a C11 atomic variable.

```
#include <stdatomic.h> atomic_long
frame_count = 0;
/* Inside each worker, after processing a frame: */
atomic_fetch_add(&frame_count, 1);
```

3.7 Process alternative and complexity

Using several processes gives each worker a separate address space and stronger fault isolation. However, creating processes and switching between them generally costs more, and results must be combined using pipes, sockets, message queues or shared memory. Threads share memory and can update a common result directly, but require synchronisation.

Approach	Memory and communication	Best use
Threads	Shared address space; lower communication overhead; shared variables require locks	High-throughput frame processing where workers use the same libraries and data.
	or atomics.	

Processes	Separate address spaces; IPC is needed to exchange results.	Untrusted or crash-prone camera decoders requiring stronger isolation.

For this workload, use a pool sized near the available CPU capacity, then benchmark 4, 8 and 16 workers. Do not automatically choose the largest pool; select the size with the best measured throughput and acceptable latency.

Q4 – INTER-PROCESS COMMUNICATION (6 MARKS)

4.1 IPC problem and communication models

The weather station has a sensor-reader process and an analyser process. They must transfer readings continuously without losing data. Inter-process communication (IPC) allows separate processes to exchange information.

Method	Communication model	Data movement and synchronisation
Pipe	Kernel-managed byte stream between related processes.	Data is copied from the writer's user buffer into a kernel pipe buffer and from the pipe buffer into the reader's user buffer. Blocking read/write provides basic flow control.
Shared memory	Processes map the same memory region.	After setup, data can be accessed without copying every reading through the kernel, but explicit synchronisation is required to avoid races.
Message queue	Kernel-managed messages with message boundaries and optional priorities.	The kernel transfers message data and manages queueing; send/receive operations provide coordination.

4.2 Pipe implementation – completed TODO

```
/* q4_pipe.c */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
int fd[2];
   if (pipe(fd)==-1) { perror("pipe"); return 1; }
pid_t pid=fork();
   if (pid<0) { perror("fork"); return 1; }
if (pid==0) {         close(fd[1]);
      int r, count=0, sum=0, max=0;
      ssize_t n;
      while ((n=read(fd[0],&r,sizeof r))>0) {
if (n != (ssize_t)sizeof r) {
            fprintf(stderr,"incomplete reading\n"); _exit(2);
         }
```

```
        count++; sum+=r;
        if (count==1 || r>max) max=r;
      }
      if (n<0) { perror("read"); _exit(1); }
      printf("analyser: %d readings, average %.1f, max %d\n",
count,count?(double)sum/count:0.0,max);        close(fd[0]);
fflush(stdout); _exit(0);
   }
close(fd[0]);
   int readings[]={31,33,36,34,41,38};
for (int i=0;i<6;i++)
      if (write(fd[1],&readings[i],sizeof readings[i])<0)
perror("write");
   close(fd[1]);              /* EOF is delivered after all writers close */
waitpid(pid,NULL,0);     puts("reader: done");     return 0; }
```

Compile and run: gcc -std=c11 -Wall -Wextra q4_pipe.c -o q4_pipe && ./q4_pipe

Expected output from the assignment: analyser: 6 readings, average 35.5, max 41; reader: done.

4.3 Why closing the pipe is essential

A pipe reader receives end-of-file (read() returns 0) only when every file descriptor referring to the pipe's write end has been closed. If the parent does not close fd[1], the child continues waiting because it cannot know that more readings will not arrive. The parent may then wait for the child, creating a hang.

4.4 Shared-memory alternative using POSIX shared memory and semaphore

A shared-memory implementation can store a fixed array of readings and a count in a mapped region. A process-shared semaphore can signal that the producer has finished. This small example demonstrates the principle for a single producer and a single consumer; production systems should also handle partial startup, errors and object cleanup.

```
/* q4_shared.c - POSIX shared memory + process-shared semaphore */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

struct shared {
sem_t ready;
int count;     int
values[6];
};
int main(void) {
   const char *name="/weather_ipc_demo";     int
fd=shm_open(name,O_CREAT|O_RDWR,0600);     if
(fd<0) { perror("shm_open"); return 1; }     if
(ftruncate(fd,sizeof(struct shared))<0) {
perror("ftruncate"); return 1;     }
```

```
   struct shared *s=mmap(NULL,sizeof *s,PROT_READ|PROT_WRITE,
                         MAP_SHARED,fd,0);
   if (s==MAP_FAILED) { perror("mmap"); return 1; }      s-
>count=0;
   if (sem_init(&s->ready,1,0)<0) { perror("sem_init"); return 1; }


   pid_t pid=fork();     if (pid==0)
{         sem_wait(&s->ready);
int sum=0,max=0;          for (int
i=0;i<s->count;i++) {
sum+=s->values[i];
           if (i==0 || s->values[i]>max) max=s->values[i];
       }
       printf("analyser: %d readings, average %.1f, max %d\n",
s->count,s->count?(double)sum/s->count:0.0,max);
_exit(0);
   }
   int data[]={31,33,36,34,41,38};      s-
>count=6;
   for (int i=0;i<6;i++) s->values[i]=data[i];
sem_post(&s->ready);      waitpid(pid,NULL,0);
sem_destroy(&s->ready);
   munmap(s,sizeof *s); close(fd); shm_unlink(name);
puts("reader: done");      return 0;
}
```

Compile and run: gcc -std=c11 -Wall -Wextra q4_shared.c -o q4_shared -pthread -lrt && ./q4_shared

The semaphore is process-shared because sem_init() uses pshared=1. The parent fills the shared data before posting the semaphore; the child waits before reading it. For a continuous producer-consumer stream, use a ring buffer with separate empty/full semaphores and a mutex, as appropriate.

4.5 Growth investigation and measurement

The assignment asks for transfer tests of 1 MB, 10 MB and 100 MB. Use the same data pattern, buffer size and machine conditions for each IPC method. Run each test several times and record the median or average.

Data size	Pipe time (s)	Shared-memory time (s)	Message-queue time (s)
1 MB	Measure	Measure	Measure
10 MB	Measure	Measure	Measure
100 MB	Measure	Measure	Measure

Suggested measurement: use clock_gettime(CLOCK_MONOTONIC) around the transfer loop. Repeat at least five times. The values are hardware- and kernel-dependent and must be filled from actual execution.

4.6 Complexity and recommendation

For n bytes, both methods perform O(n) data work. A pipe using a buffer of b bytes requires approximately O(n/b) read/write operations, plus data copying through the kernel. Shared memory can avoid repeated kernel-mediated data copies after mapping, but still requires O(n) memory access and synchronisation. Shared memory is usually preferable for very high-volume local data exchange when low latency is important. Pipes are simpler and safer for a modest stream of readings.

Q5 – SYNCHRONISATION: CRITICAL SECTIONS, SEMAPHORES AND MONITORS (11 MARKS)

5.1 Critical-section problem

A critical section is the part of a program that accesses shared data, such as the available-seat counter in a railway reservation system. A correct solution must satisfy three requirements:

- Mutual exclusion: at most one process or thread at a time enters the critical section for the protected resource.
- Progress: if no process is in the critical section and some wish to enter, the selection of the next process cannot be postponed indefinitely.
- Bounded waiting: after a process requests entry, there is a limit on the number of times others may enter before it.

5.2 Peterson's solution

Peterson's algorithm is a software solution for two processes. It uses flag[2] to show interest and turn to resolve simultaneous requests. Each process raises its flag, gives the other process priority, and waits while the other is interested and it is the other process's turn. Under the classical sequentially consistent memory model, it satisfies mutual exclusion, progress and bounded waiting.

On modern processors and compilers, memory operations may be reordered or observed differently because of caches and relaxed memory models. A plain implementation with ordinary variables is therefore not a safe general-purpose lock in modern C. Atomic variables with suitable memory ordering or, preferably, POSIX mutexes should be used.

5.3 Binary and counting semaphores

Semaphore	Meaning	Railway example
Binary semaphore	Value is 0 or 1; used for exclusive access or event signalling.	Protect one shared seat counter or critical section.
Counting semaphore	Non-negative count represents available instances of a resource.	Track the number of unreserved seats or free booking slots.

wait (also called P/down) decrements the semaphore when a resource is available, otherwise the caller blocks. signal (also called V/up) increments the semaphore and may wake a waiting thread. A

mutex protects the actual buffer indices and data; semaphores represent available and occupied buffer slots.

5.4 Classical synchronisation problems

Bounded-buffer (producer-consumer): use empty initialised to buffer capacity, full initialised to zero and a mutex for the buffer. A producer waits on empty, inserts an item under the mutex and signals full. A consumer waits on full, removes an item under the mutex and signals empty.

Readers-writers: multiple readers may read simultaneously, but a writer requires exclusive access. A reader count is protected by a mutex; the first reader locks the resource and the last reader unlocks it. Writer preference or fair queuing may be added to prevent starvation.

Dining philosophers: each philosopher needs two adjacent forks. Naively taking one fork and waiting for the other can create circular wait. Break the cycle by ordering fork acquisition, allowing at most four philosophers to compete simultaneously, or using a monitor that grants both forks together.

5.5 Monitor and condition variables

A monitor groups shared data and the procedures that access it. Only one thread executes a monitor procedure at a time. Condition variables allow a thread to wait until a condition becomes true and another thread to signal it. In POSIX C, a mutex and condition variables can implement monitor-like behaviour. Condition waits must be placed in a while loop because wake-ups do not guarantee that the condition is still true.

```
/* Monitor-style bounded condition check */
pthread_mutex_lock(&mutex); while (count
== 0)
   pthread_cond_wait(&not_empty, &mutex); /*
remove an item while holding the mutex */
pthread_mutex_unlock(&mutex);
```

5.6 Completed semaphore and mutex program

```
/* q5_buffer.c - completed TODO 1 and TODO 2 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>


#define SIZE 5
#define ITEMS 20 int
buffer[SIZE], in=0, out=0;
sem_t empty, full;
pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t sum_lock=PTHREAD_MUTEX_INITIALIZER; long
consumed_sum=0;

void *producer(void *arg) {
```